WebAssembly Memory Tagging

Shengdun Wang

20241125

Advisor

Aravind Prakash

Background

- The web IS the predominant platform for app development (All Win11 preinstalled apps are web apps).
- Everything Old is New Again: Binary Security of WebAssembly

Why WebAssembly?

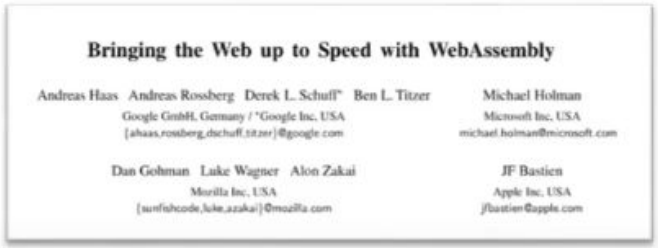
- JavaScript shortcomings
  - Source code format: slow parsing, large code size
  - Relies on JIT compilation and GC: performance not reliable
  - Only available language on the client-side
    - Abused as a compilation target, "transpilation" of CoffeeScript etc. → JavaScript
- Other technologies in the past
  - ActiveX, Java applets, Flash: (host) security nightmare
  - NaCl and PNaCl: only Chrome
  - asm.js: subset of JavaScript

```
function f(i) {
  i = i|0;
  return (i + 1)|0;
}
```

WebAssembly Timeline

- Announced in 2015
- First support in all major browsers in 2017

- Haas et al., PLDI 2017
- Version 1 standardized by W3C in 2019



Bringing the Web up to Speed with WebAssembly

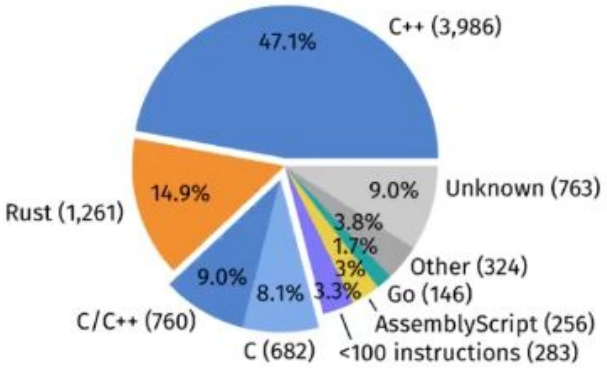
Andreas Haas Andreas Rossberg Derek L. Schuff* Ben L. Titzer Michael Holman
Google GmbH, Germany / *Google Inc, USA Microsoft Inc, USA
{ahaas,rossberg,dschuff,titzer}@google.com michael.holman@microsoft.com

Dan Gohman Luke Wagner Alon Zakai JF Bastien
Mozilla Inc, USA Apple Inc, USA
{sunfishcode,luke,azakai}@mozilla.com jfbastien@apple.com

- Today
  - 50.000 uses in top 1M websites
  - Widely supported, also on mobile

Ecosystem

- Languages with WebAssembly backend
  - Emscripten, clang: C/C++
  - Rust
  - AssemblyScript (subset of TypeScript)
  - Go, and several others...
- Host environments
  - Browsers: Firefox, Chrome, Safari, Edge
  - Server-side: Node.js, Cloudflare workers
  - Stand-alone VMs: wasmtime, Lucet, wasmer, WAVM, ...
    - WASI: WebAssembly Systems Interface
    - Smart contract systems: Ethereum (eWASM), EOS.IO



WIP: study of real-world WebAssembly

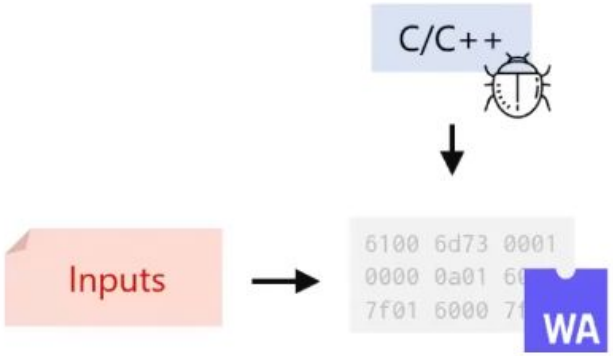
Two Aspects to Security

- Separate memory
- Validation of binaries
- Browser sandbox

Cryptomining

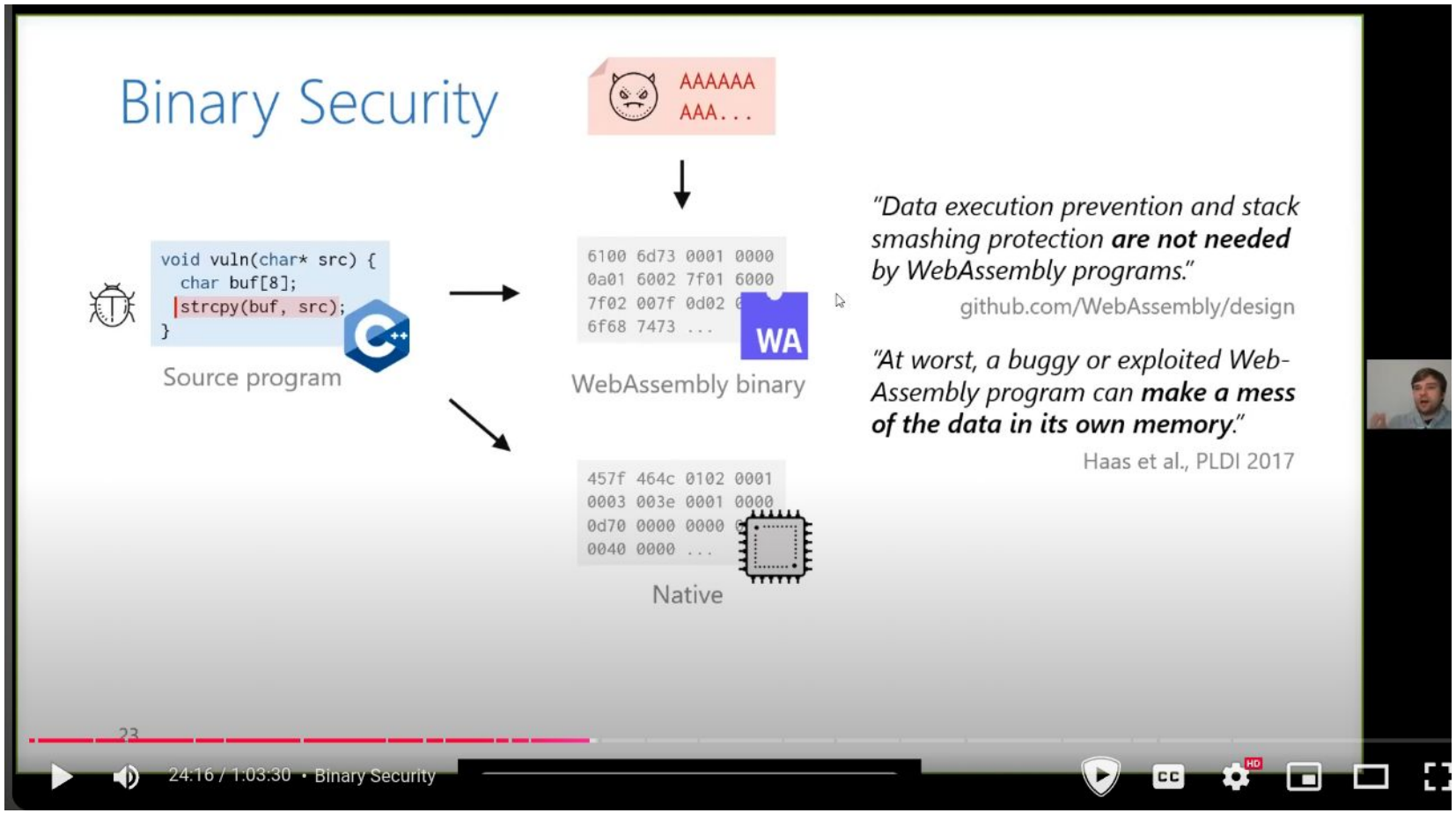
Security of the **host**

Malicious WebAssembly



Security of the **binary**

Vulnerable WebAssembly


Binary Security
AAAAAA
AAA...
void vuln(char* src) {
char buf[8];
strcpy(buf, src);
}
Source program
6100 6d73 0001 0000
0a01 6002 7f01 6000
7f02 007f 0d02
6f68 7473 ...
WA
WebAssembly binary
457f 464c 0102 0001
0003 003e 0001 0000
0d70 0000 0000
0040 0000 ...
Native
"Data execution prevention and stack smashing protection **are not needed** by WebAssembly programs."
github.com/WebAssembly/design
"At worst, a buggy or exploited Web-Assembly program can **make a mess of the data in its own memory**."
Haas et al., PLDI 2017
23
24:16 / 1:03:30 • Binary Security

https://nvd.nist.gov/vuln/detail/CVE-2021-31162

VULNERABILITIES

CVE-2021-31162 Detail

MODIFIED

This vulnerability has been modified since it was last analyzed by the NVD. It is awaiting reanalysis which may result in further changes to the information provided.

Description

In the standard library in Rust before 1.52.0, a double free can occur in the Vec::from_iter function if freeing the element panics.

Metrics

CVSS Version 4.0 | CVSS Version 3.x | CVSS Version 2.0

NVD enrichment efforts reference publicly available information to associate vector strings. CVSS information contributed by other sources is also displayed.

CVSS 3.x Severity and Vector Strings:

NIST: NVD **Base Score:** 9.8 CRITICAL **Vector:** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

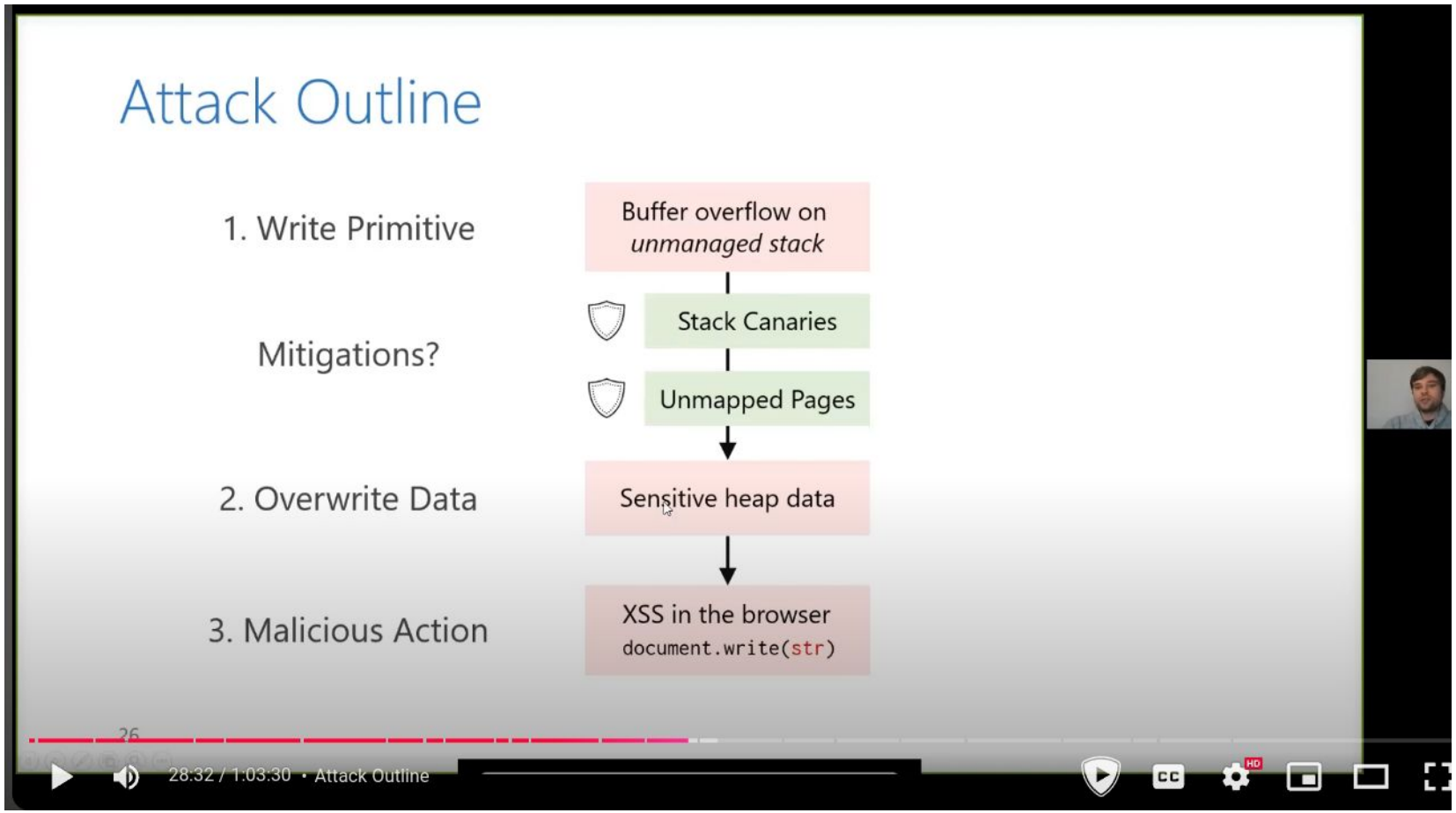
QUICK INFO

CVE Dictionary Entry:
CVE-2021-31162
NVD Published Date:
04/14/2021
NVD Last Modified:
11/06/2023
Source:
MITRE

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Hyperlink	Resource
https://github.com/rust-lang/rust/issues/83618	Exploit Third Party Advisory
https://github.com/rust-lang/rust/pull/83629	Patch Third Party Advisory


Attack Outline
1. Write Primitive
Buffer overflow on
unmanaged stack
Mitigations?
Stack Canaries
Unmapped Pages
2. Overwrite Data
Sensitive heap data
3. Malicious Action
XSS in the browser
document.write(str)
26
28:32 / 1:03:30 • Attack Outline

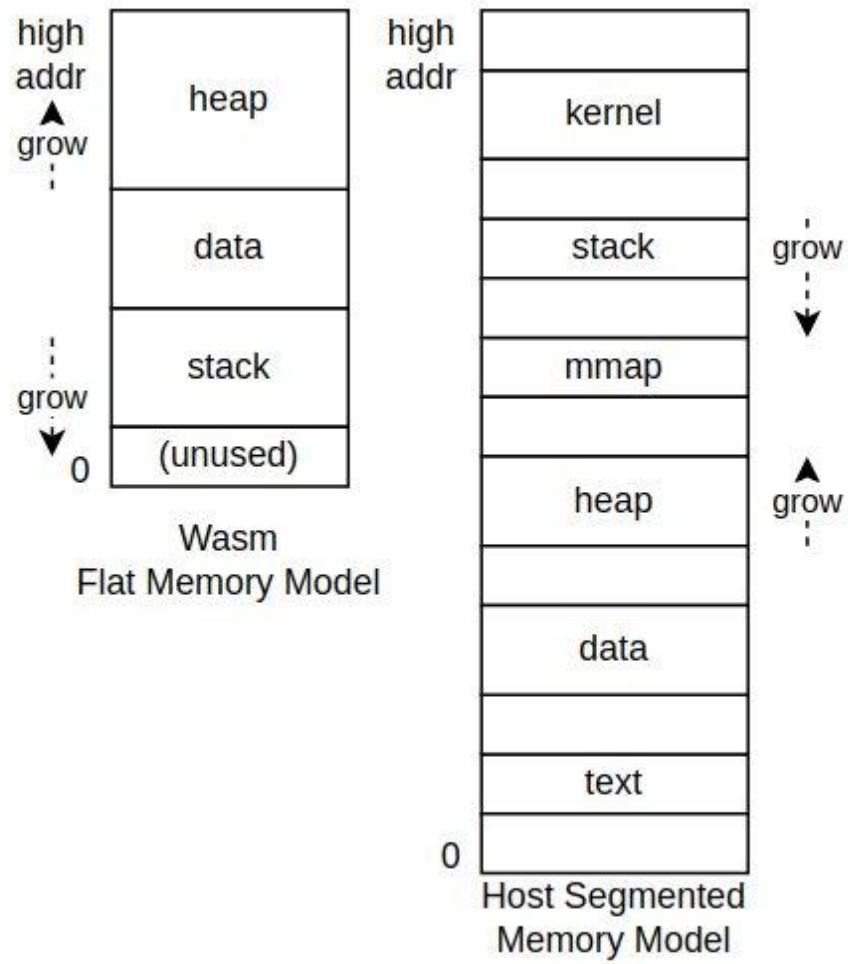
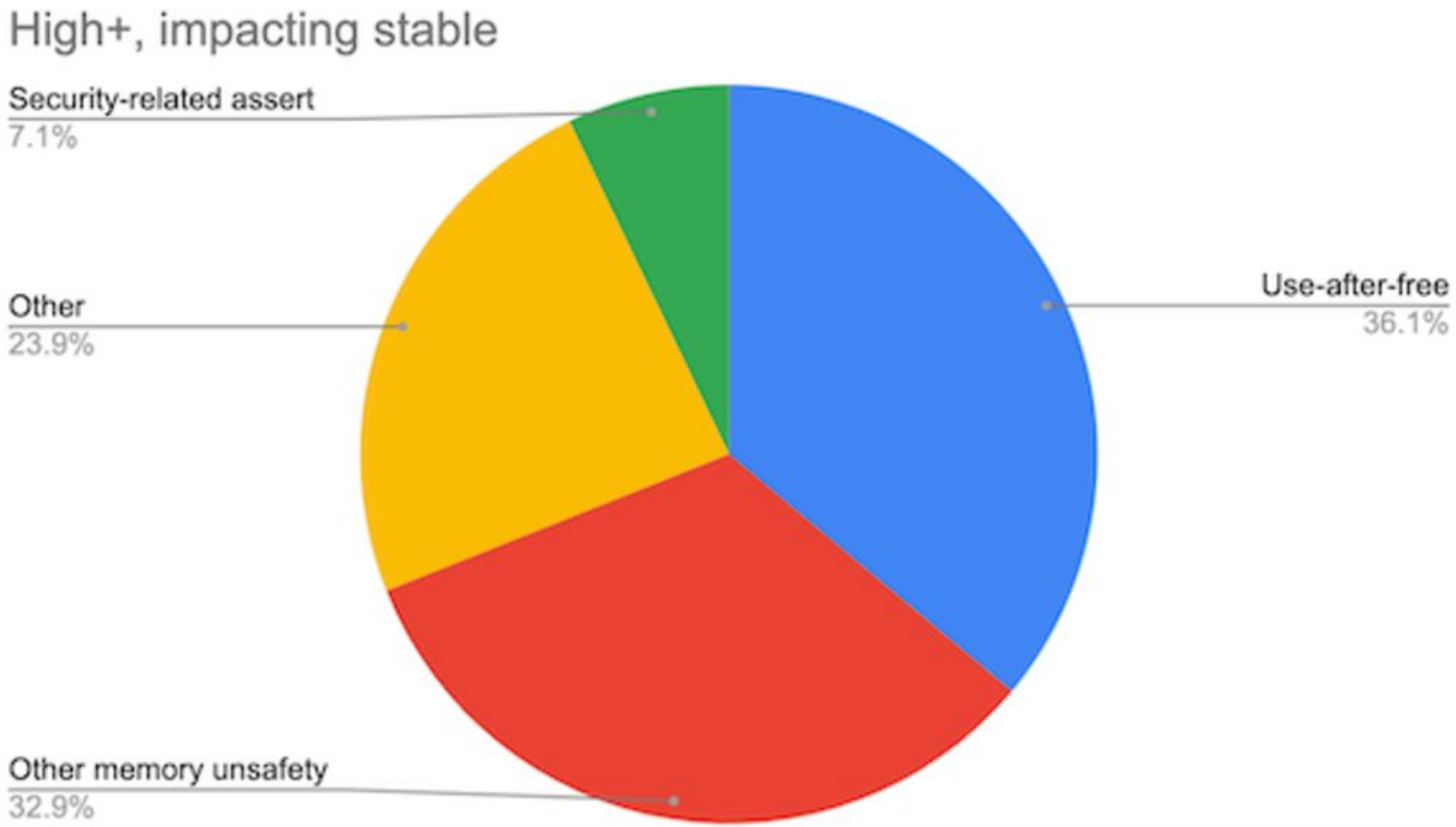


Figure 1: Wasm vs Host Memory Model

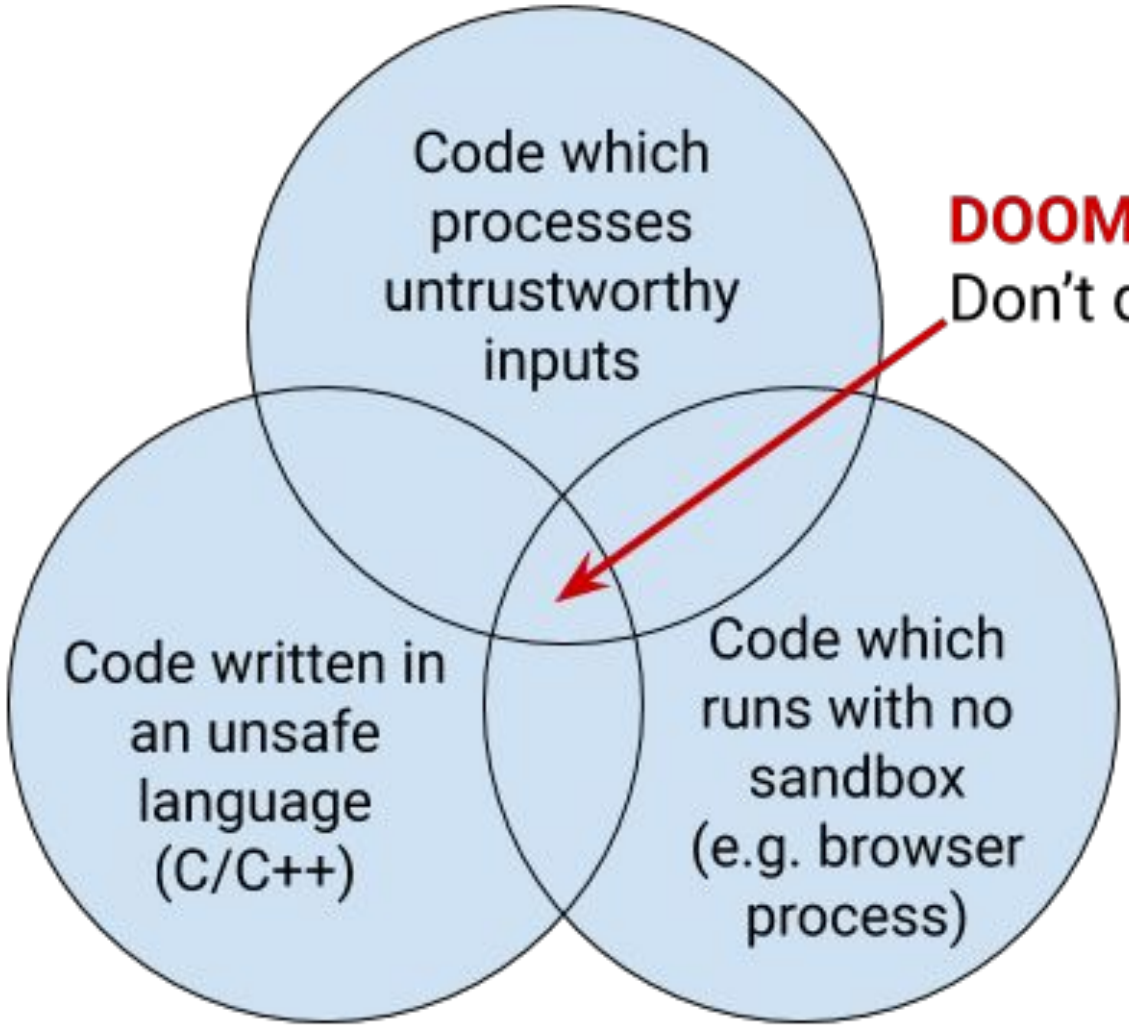
The problem

Around 70% of our high severity security bugs are memory unsafety problems (that is, mistakes with C/C++ pointers). Half of those are use-after-free bugs.



(Analysis based on 912 high or critical severity security bugs since 2015, affecting the Stable channel.)

“Rule of Two”



DOOM!
Don’t do this.

High Level Idea ---Practical

Practical

Proven technique in hosted environment

Goal: Rule of 2

Memory Safety

The Chromium project finds that around 70% of our serious security bugs are memory safety problems. Their next major project is to prevent such bugs at source.

The problem

Around 70% of their high severity security bugs are memory unsafety problems (that is, mistakes with C/C++ pointers). Half of those are use-after-free bugs.

Static Analyzer?

- They have limitations.

Address Sanitizers?

- It is not a hardening tool.
- Wasm unsupported now.

ASLR, DEP?

- How do you implement pages through software efficiently?

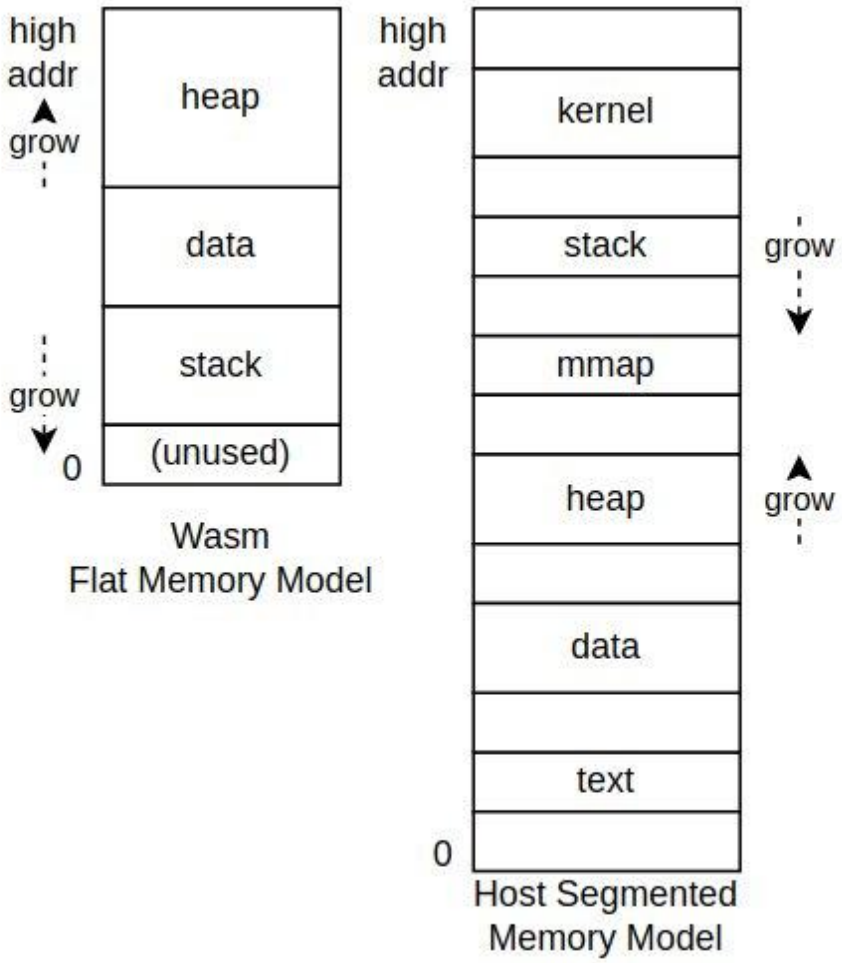


Figure 1: Wasm vs Host Memory Model

Already this work is having a positive impact. Kuaishou – a leading content community and social platform for video sharing and live streams with over 360 million daily average users and 626 million monthly average users, making it the second-largest short video platform in the world – is partnering with Honor SkyNet and using Arm MTE to enhance the efficiency of memory safety across the development cycles of large-scale software projects. This has led to 90 percent of memory bugs being detected before release, alongside the following additional improvements:

+ 3x scan speed improvement when scanning for memory bugs before app deployment
+ Native RAM usage decreasing by 50 percent
+ Ten serious bugs being found, with this not being possible without MTE.

Here's what Kuaishou – whose overseas products, Kwai and SnackVideo, cover more than 30 countries and 160 million users – had to say about MTE:

> *"At Kuaishou, our mission is to be the most customer-obsessed company in the world. Therefore, we place a high emphasis on ensuring the ultimate user experience, with privacy and safety a significant part of this commitment. Kuaishou's R&D team has committed serious efforts to ensuring a safe and secure user experience, with MTE playing a significant role in guaranteeing memory safety across the platform. Due to the high performance overhead of traditional memory sanitizer tools and the requirement of re-compiling all of the source code, it is almost impossible to use these tools in*

MTE - The promising path forward for memory safety

November 7, 2023

Posted by Andy Qin, Irene Ang, Kostya Serebryany, Evgenii Stepanov

Since 2018, Google has partnered with ARM and collaborated with many ecosystem partners (SoCs vendors, mobile phone OEMs, etc.) to develop Memory Tagging Extension (MTE) technology. We are now happy to share the growing adoption in the ecosystem. MTE is now available on some OEM devices (as noted in a recent blog post by Project Zero) with Android 14 as a developer option, enabling developers to use MTE to discover memory safety issues in their application easily.

The security landscape is changing dynamically, new attacks are becoming more complex and costly to mitigate. It's becoming increasingly important to detect and prevent security vulnerabilities early in the software development cycle and also have the capability to mitigate the security attacks at the first moment of exploitation in production.

The biggest contributor to security vulnerabilities are memory safety related defects

ARM MTE

- ARM MTE divides memory address space into 16-byte granules, each associated with a 4-bit tag.
- It has two tags: pointer tags and address tags. The bits 59-56 of the pointer are used as a color and ignored for addressing.
- Detection is probabilistic: The Chance of detection is $1-2^{-n}$

  (ARM MTE) n=4. The detection rate is $1-2^{-4} = 93.75\%$

  If n=8. The detection rate is $1-2^{-8}=99.61\%$

The problem of ARM MTE

It lacks adoption. Microsoft and Apple's flagship product.

Surface Pro 11 (Qualcomm Snapdragon X Elite Copilot+ PC) and Apple M4 do not support ARM MTE.

In android world, only Google Pixel 8 supports ARM MTE.

This is an ARM only feature. It does not support x86, riscv, loongarch etc.

WAVM

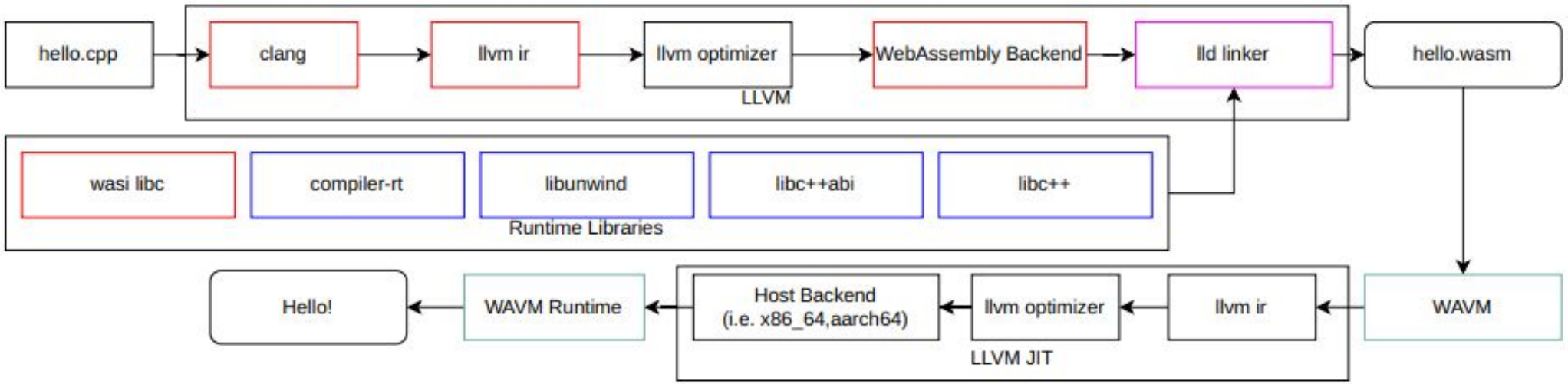


Figure 3.3: Wasm Workflow: C++ code compiled by LLVM into Wasm, then WAVM runs the Wasm with LLVMJIT

Memory Tagging Mechanism

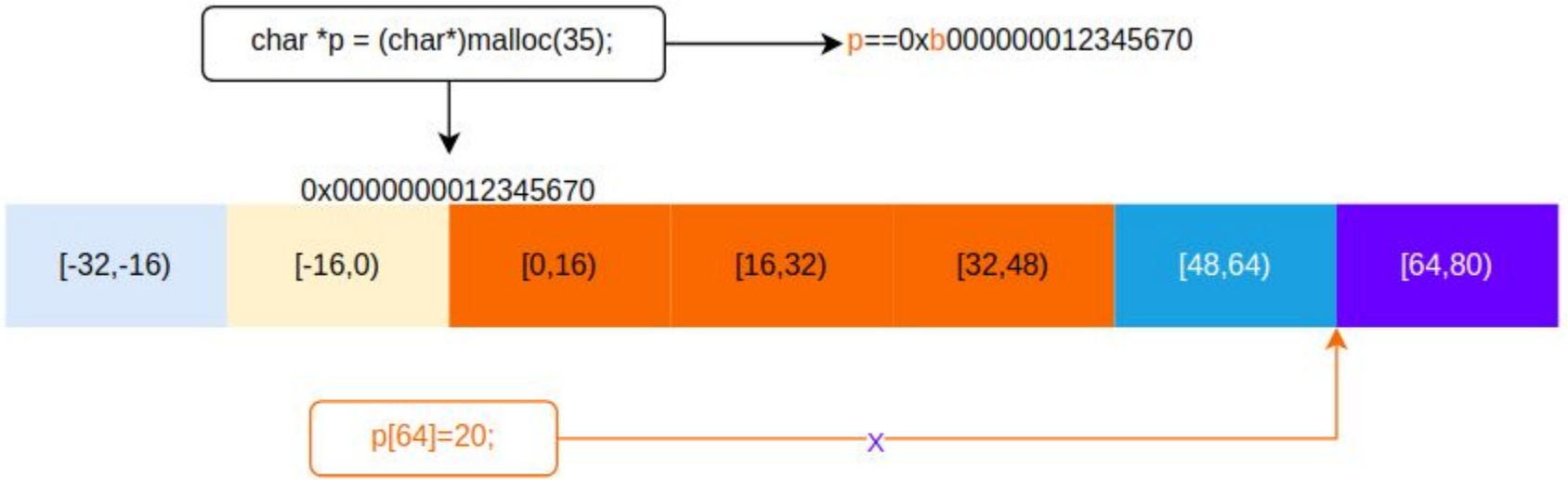


Figure 3.2: Memory Tagging Mechanism: Tag on the pointer does not match the memory color on p+64. The program crashes

WebAssembly Memory Tagging

- WebAssembly's Flag Memory Model makes it simple to implement memory tagging mechanism through software emulation.
- The paper chose to align the behavior of WebAssembly Memory Tagging with ARM MTE.
- The paper added new WebAssembly Instructions into LLVM backend. The granule is 16 bytes just like ARM MTE.
- The paper modified WAVM (an efficient LLVM based Wasm Virtual Machine) to support the memory tagging as the demo.
- The tag is stored in another shadow memory which is not indexable directly by users.
- Memory of Null Address is tagged with a none-zero tag

Table 3.1: Part of WebAssembly Memory Tagging Instructions

Instruction	Description
`memtag.random`	Similar to ARM MTE's IRG instruction, this instruction generates a random tag for an untagged pointer, tags the upper bits, and returns the tagged pointer.
`memtag.randommask`	Provide an extra mask flag for the tag, similar to mask in ARM MTE's IRG
`memtag.load`	Loads the tag from the memory address pointed to by the pointer and tag the pointer.
`memtag.sub`	Subtracts two pointers by disregarding their tag bits.
`memtag.copy`	Copies the upper tag bits from the source pointer to the upper bits of the destination pointer.
`memtag.store`	Stores the tag of the pointer into the memory address pointed to by the pointer, using the upper bits of the pointer as the tag. This instruction also accepts an argument `b16` to specify the number of 16-byte segments to be tagged. The granule size is set to 16; the number provided must be a multiple of 16. If the provided number is not a multiple of 16, it will tag the integer division by 16 and then multiply by 16. Unlike ARM MTE, which only stores tags for 16 bytes per instruction, `memtag.store` can store arbitrary numbers of bytes, and the byte parameter does not need to be a constant, making it more versatile and easier to implement for compilers.
`memtag.storez`	Similar to `memtag.store`, but it also zeros the memory.
`memtag.randomstore`	A combined operation of `memtag.random` and `memtag.store`. This instruction randomly generates a tag, tags both the memory address pointed to by the pointer, and returns the pointer with the new tag. It also accepts `b16` as a parameter to tag arbitrary numbers of bytes in memory.
`memtag.randomstorez`	Similar to `memtag.randomstorez`, but it also zeros the memory it tags.
`memtag.hint`	Uses the tag of the source pointer and an index as a hint to tag the new pointer. Compared to `memtag.random`, the VM can implement the new pointer's tag by adding the index to the source pointer's tag. This allows for generating a new tag without random number generation and avoids duplicate tags. However, this is only a hint; the VM can ignore it if necessary.
`memtag.hintstore`	Similar to `memtag.randomstore`, but it uses the source pointer as a hint and the index as the tag without regenerating the tag to tag the pointer.
`memtag.hintstorez`	Similar to `memtag.hintstore`, but it also zeros the memory.

- let hasSideEffects = 1, mayStore = 1 in
- defm MEMTAG_RANDOMSTORE_A#B: I<(outs rc:$dst), (ins i32imm:$tableidx, rc:$src, rc:$b16),
- (outs), (ins i32imm:$tableidx),
- [(set rc:$dst,
- (int_wasm_memtag_randomstore (i32 imm:$tableidx), rc:$src, rc:$b16))],
- "memtag.randomstore\t$dst, $tableidx, $src, $b16",
- "memtag.randomstore\t$tableidx",
- 0xfc2e>;

- let hasSideEffects = 1, mayStore = 1 in
- defm MEMTAG_RANDOMSTOREZ_A#B: I<(outs rc:$dst), (ins i32imm:$tableidx, rc:$src, rc:$b16),
- (outs), (ins i32imm:$tableidx),
- [(set rc:$dst,
- (int_wasm_memtag_randomstorez (i32 imm:$tableidx), rc:$src, rc:$b16))],
- "memtag.randomstorez\t$dst, $src, $tableidx, $b16",
- "memtag.randomstorez\t$tableidx",
- 0xfc2f>;

b16

- The meaning of **b16**:
- It is **not literally 16**
- It means “align this value down to a 16-byte boundary”
- Implemented as: `(b16 >> 4) << 4`
- Examples:
  - 15 → 0
  - 16 → 16
  - 17 → 16
  - 31 → 16

How many bits to tag?

The VM implementation is free to decide how many bits to tag.

In our implementation we choose 8 bits tag for 64 bit Wasm, 4 bits tag for 32 bit Wasm.

32bit Wasm, n=4. The detection rate is $1-2^{-4} = 93.75\%$ (side effect: 4 bit tags reduces the usable memory to 256MB)

64bit Wasm, n=8. The detection rate is $1-2^{-8} = 99.61\%$

Todo: Automatically translate Wasm Memtag instruction to ARM MTE supported machines. (Hardware implementation)

- memtag.add Adds an address offset and a tag offset to a tagged pointer. Takes a table index, a source pointer, an address offset, and a tag offset, and returns the updated pointer. Semantics: int_wasm_memtag_add(tableidx, src, addroffset, tagoffset)
- memtag.addstore Performs memtag.add and stores the result. This instruction may store. b16 is aligned down to a 16-byte boundary. Semantics: int_wasm_memtag_addstore(tableidx, src, aligned_b16, addroffset, tagoffset)
- memtag.addstorez Zero-extended variant of memtag.addstore. b16 is aligned down to a 16-byte boundary. Semantics: int_wasm_memtag_addstorez(tableidx, src, aligned_b16, addroffset, tagoffset)
- memtag.hint Provides a hint to the tagging engine, such as a prefetch-like advisory. Takes a table index, a source pointer, a hint address, and an index. Semantics: int_wasm_memtag_hint(tableidx, src, hintaddr, index)
- memtag.hintstore Performs a hint operation and stores a value. This instruction may store. b16 is aligned down to a 16-byte boundary. Semantics: int_wasm_memtag_hintstore(tableidx, src, aligned_b16, hintaddr, index)
- memtag.hintstorez Zero-extended variant of memtag.hintstore. b16 is aligned down to a 16-byte boundary. Semantics: int_wasm_memtag_hintstorez(tableidx, src, aligned_b16, hintaddr, index)

Memtag.add

- Compared to memtag.add, the hint mechanism is generally more appropriate. ARM MTE includes an instruction that adds a tag offset directly to a pointer’s tag, but this approach is not ideal for WebAssembly memtag. WebAssembly does not want to force users or ABIs to rely on a specific tag arithmetic model, and the number of tag bits used by the VM is environment-dependent. The tag width is intentionally agnostic. Because of this, memtag.add and related addstore instructions exist only as placeholders. They are not intended to enforce a particular tag-arithmetic model, and they do not assume any specific number of tag bits. The hint mechanism is the preferred way to express tag-generation intent without constraining the VM’s implementation choices.

ABI Compatible

No break of existing code. Even relinking of newly compiled libc would get it.

The VM is free to ignore the tagging (but not zeroing aka memtag.randomstorez)

https://github.com/llvm/llvm-project/pull/162972

Instructions started with prefix memtag.

- memtag.status
- memtag.random

How the tag is generated?

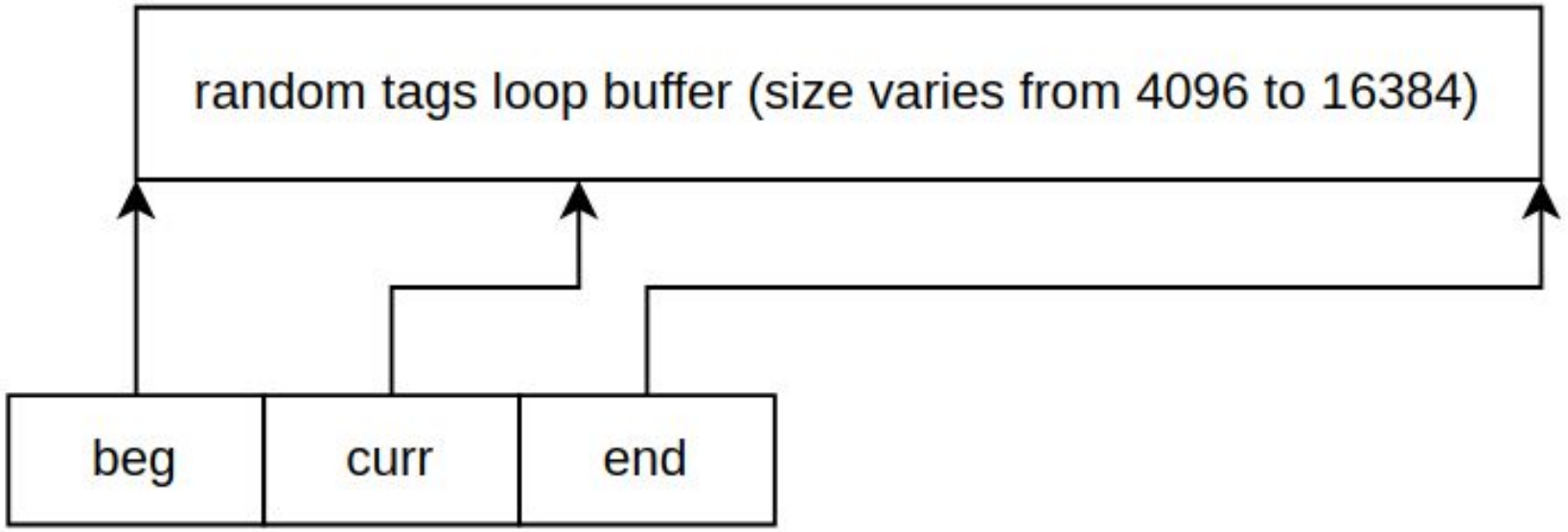


Figure 3.4: Memtag Loop Buffer: The tag buffer is 4k to 16k, the curr ptr loops the tag buffer to fetch tags

Release mode and debug mode

Listing 3.1: Wasm Memory Tagging False Negative

```
1 char buffer[16];
2 int v;
3 //Release mode does not detect this but debug mode can
4 memcpy(&v, buffer+15, sizeof(v));
```

Evaluation

Table 1. Benchmark Results (Overhead over Wasm Baseline) on x86_64-linux-gnu

	64-bit		32-bit	
	Release(%)	**Debug(%)**	**Release(%)**	**Debug(%)**
CHStone	34.21	65.92	82.44	132.31
PolyBench/C	88.98	249.30	135.26	308.11
SQLite3	28.76	58.75	20.05	49.56
fastiobench	43.67	71.02	65.47	103.26
Average	48.91	112.47	72.38	143.36

Figure 5. Benchmarks on x86_64-linux-gnu (Larger is Slower, Wasm64 (Baseline) is 1)

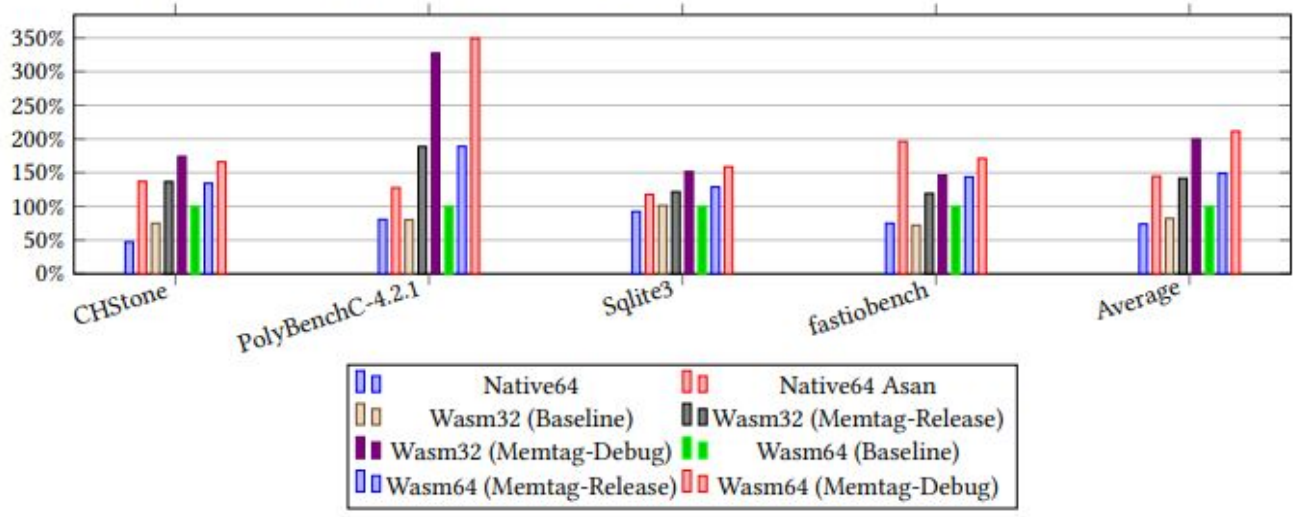


Table 2. Memory Safety Effectiveness Evaluation

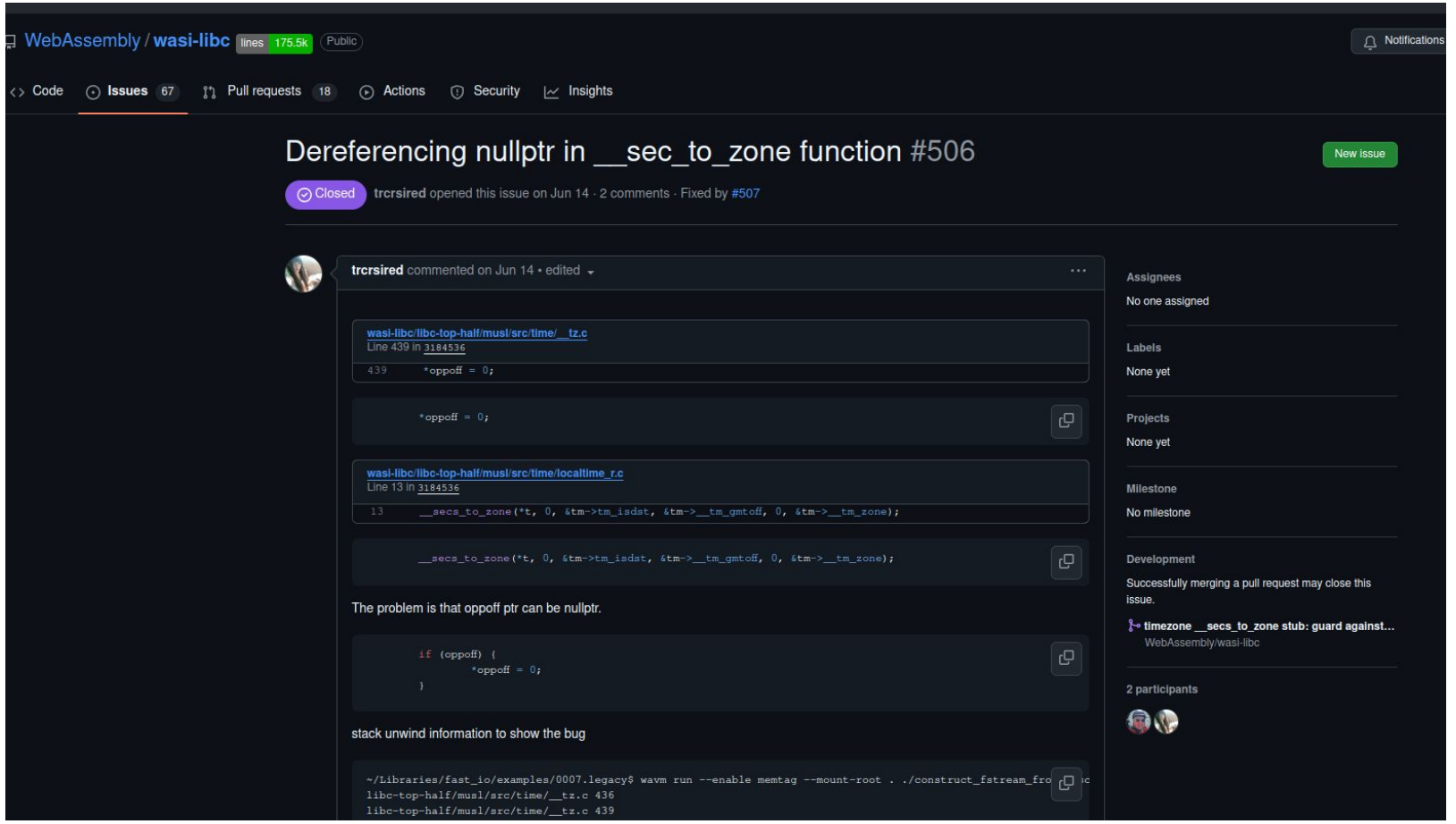
	Native	Native ASan	Wasm (Baseline)	Wasm (Memtag)
allocation-size-too-big	✓ (Segfault)	✔	✕	✓
alloc-dealloc-mismatch	✕	✕	✕	✕
calloc-overflow	✕	✔	✕	✓
container-overflow	✕	✔	✕	✔
double-free-operator-delete	✔ (glibc detected)	✔	✕	✔
double-free-free	✔ (glibc detected)	✔	✕	✔
memcpy-param-overlap	✕	✔	✕	✕
new-delete-type-mismatch	✕	✕ (Leak sanitizer)	✕	✕
dynamic-stack-buffer-overflow	✕	✔	✕	✕
global-buffer-overflow	✕	✔	✕	✕ (Todo)
stack-buffer-overflow	✕	✕	✕	✔✕
stack-buffer-math	✕	✕	✕	✔✕
heap-buffer-overflow	✕	✔	✕	✔✕
heap-use-after-free (4 tests)	✕	✔ (4/4)	✕	✔ (4/4)
invalid-allocation-alignment	✕	✔	✕	✔
stack-use-after-return	✕	✔	✕	✔
stack-use-after-scope (4 tests)	✕	✔	✕	✕
stack-use-after-scope-fn (4 tests)	✕	✔ (4/4)	✕	✔ (4/4)
strcat-param-overlap	✕	✔	✕	✕
nullptr-dereference	✓ (Segfault)	✓	✕	✔

Wasm Memtag can, Asan cannot

Listing 6. Dangling string_view

```
1  fast_io::string str("helloworld");
2  fast_io::string_view vw(str.subview(3, 4));
3  println("before replace:", str, "\n", vw);
4  str.replace_index(1, 5, vw);
5  // dangling string_view
6  // ASan cannot detect this, but our Wasm memtag can
7  println("after replace:", str, "\n", vw);
```

Real World Bug Found



trcrsired opened this issue on Jun 14 · 2 comments · Fixed by #507

stack unwind information to show the bug

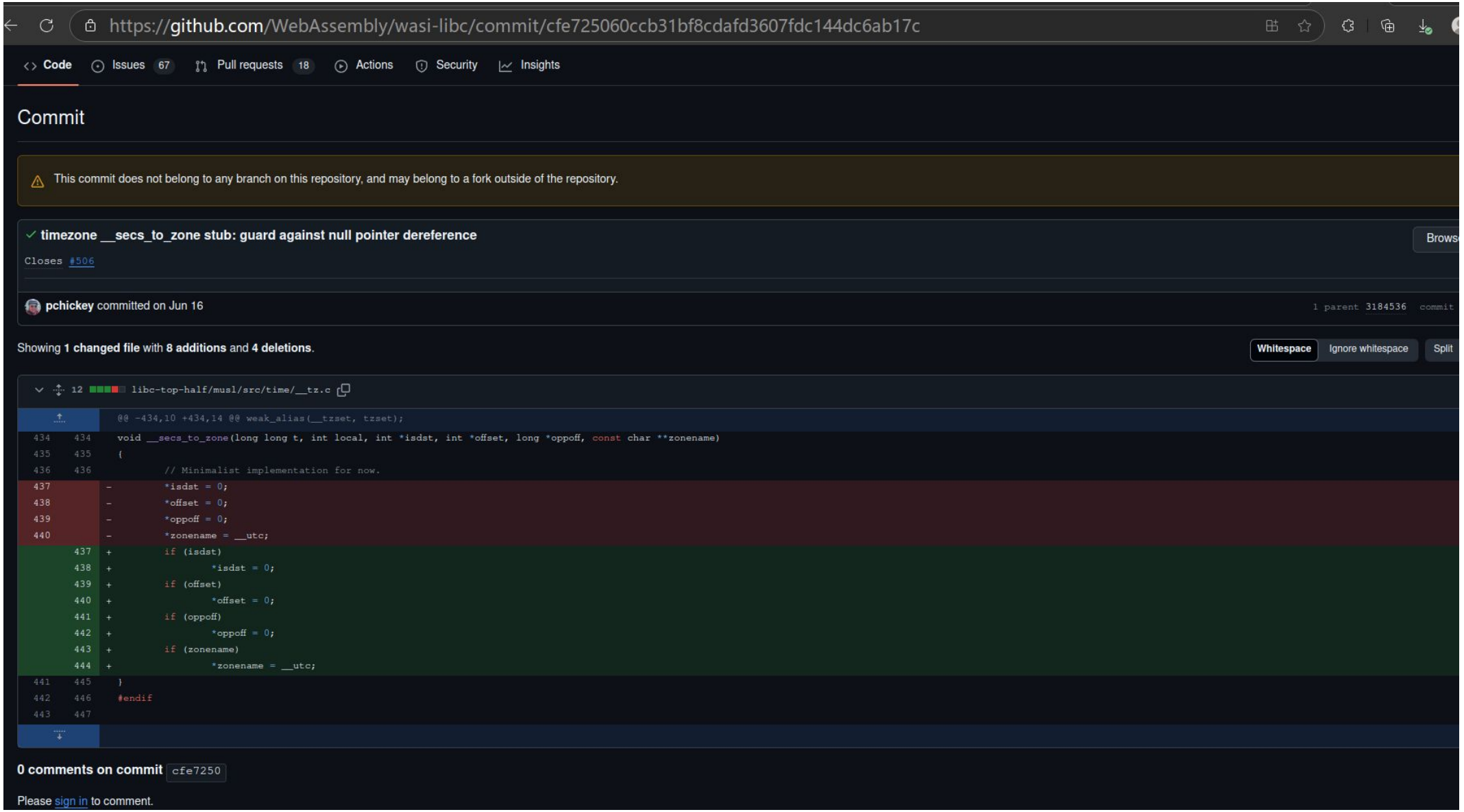
```
~/Libraries/fast_io/examples/0007.legacy$ wavm run --enable memtag --mount-root . ./construct_fstream_fro c
libc-top-half/musl/src/time/__tz.c 436
libc-top-half/musl/src/time/__tz.c 439
libc-top-half/musl/src/time/__tz.c 441
warning: DWARF unit at offset 0x0000002a has unsupported version 0, supported are 2-5
Runtime exception: wavm.invalidMemoryTagAccess
Call stack:
  host!/softwares/wavm/lib/libWAVM.so.0!wavm_memtag_trap_function+41
  wasm!./construct_fstream_from_syscall!__secs_to_zone+7
  wasm!./construct_fstream_from_syscall!__localtime_r+22
  wasm!./construct_fstream_from_syscall!__original_main+314
  wasm!./construct_fstream_from_syscall!_start+13
  thnk!C to WASM thunk!()->()+0
  host!/softwares/wavm/lib/libWAVM.so.0!WAVM::Platform::catchSignals(void (*)(void*), bool (*)(void*, WAVM::Pla
  host!/softwares/wavm/lib/libWAVM.so.0!WAVM::Runtime::invokeFunctionWithMemTag(WAVM::Runtime::Context*, WAVM::
  host!wavm+252855
  host!/softwares/wavm/lib/libWAVM.so.0!WAVM::WASI::catchExit(std::function<int ()>&&)+15
  host!wavm+246546
  host!wavm+236837
  host!/softwares/wavm/lib/libWAVM.so.0!WAVM::Platform::catchSignals(void (*)(void*), bool (*)(void*, WAVM::Pla
  host!/softwares/wavm/lib/libWAVM.so.0!WAVM::Runtime::catchRuntimeExceptions(std::function<void ()> const&, st
  host!wavm+235740
  host!wavm+235511
  host!/usr/lib/libc.so.6+154759
  host!/usr/lib/libc.so.6+139
  host!wavm+72980

Aborted (core dumped)
```

Since wasi has basically been used for every wasm apps on the web and wasi-libc is static compiled so all binaries have to recompile.

trcrsired commented on Jun 14 • edited — Author

after patching the binary

```
:~/Libraries/fast_io/examples/0007.legacy$ clang++ -o construct_fstream_from_syscall construct_fstream_fr s
:~/Libraries/fast_io/examples/0007.legacy$ wavm run --enable memtag --mount-root . ./construct_fstream_fr s
Unix Timestamp:1718406476.713804722
Universe Timestamp:434602343147641676.713804722
UTC:2024-06-14T23:07:56.713804722Z
Local:2024-06-14T23:07:56.713804722Z Timezone:UTC
LLVM clang 19.0.0git (git@github.com:trcrsired/llvm-project.git e41af7174893dcae864e5046ea284948ae197f3b)
LLVM libc++ 190000
fstream.rdbuf():0xa0014ab4
FILE*:0xf0014d20
fd:5
```


https://github.com/WebAssembly/wasi-libc/commit/cfe725060ccb31bf8cdafd3607fdc144dc6ab17c
Code
Issues 67
Pull requests 18
Actions
Security
Insights
Commit
This commit does not belong to any branch on this repository, and may belong to a fork outside of the repository.
timezone __secs_to_zone stub: guard against null pointer dereference
Closes #506
pchickey committed on Jun 16
1 parent 3184536 commit
Showing 1 changed file with 8 additions and 4 deletions.
Whitespace
Ignore whitespace
Split
12 libc-top-half/musl/src/time/__tz.c
@@ -434,10 +434,14 @@ weak_alias(__tzset, tzset);
434 434 void __secs_to_zone(long long t, int local, int *isdst, int *offset, long *oppoff, const char **zonename)
435 435 {
436 436 // Minimalist implementation for now.
437 - *isdst = 0;
438 - *offset = 0;
439 - *oppoff = 0;
440 - *zonename = __utc;
437 + if (isdst)
438 + *isdst = 0;
439 + if (offset)
440 + *offset = 0;
441 + if (oppoff)
442 + *oppoff = 0;
443 + if (zonename)
444 + *zonename = __utc;
441 445 }
442 446 #endif
443 447
0 comments on commit cfe7250
Please sign in to comment.

Closed **Dereferencing nullptr in __sec_to_zone function** #506

trcrsired opened this issue on Jun 14 · 2 comments · Fixed by #507

pchickey added a commit that referenced this issue on Jun 16

`timezone __secs_to_zone stub: guard against null pointer dereference` ··· ✓ cfe7250

pchickey mentioned this issue on Jun 16

timezone __secs_to_zone stub: guard against null pointer dereference #507 Merged

pchickey commented on Jun 16 Collaborator ···

Thanks you for the bug report. The code you found it producing a null pointer dereference

wasi-libc/libc-top-half/musl/src/time/__tz.c
Lines 436 to 440 in 3184536

```
436     // Minimalist implementation for now.
437     *isdst = 0;
438     *offset = 0;
439     *oppoff = 0;
440     *zonename = __utc;
```

is a stub in place of the upstream musl implementation, which cannot be supported until timezone is available in new import functions are introduced in WASI 0.2.1. I submitted a fix for the stub #507.

abrown closed this as completed in #507 on Jun 17

abrown closed this as completed in ebac9ae on Jun 17

Bibliography

- [1] Serebryany, K. Arm memory tagging extension and how it improves c/c++ memory safety. The Usenix Magazine 44, 2 (2019), 12–16.
- [2] Google Online Security Blog. Mte - the promising path forward for memory safety, November 2023. Accessed: 2024-09-02.
- [3] Serebryany, K., Bruening, D., Potapenko, A., and Vyukov, D. Addresssanitizer: a fast address sanity checker. In Proceedings of the 2012 USENIX Conference on Annual Technical Conference (USA, 2012), USENIX ATC'12, USENIX Association, p. 28.
- [4] Shacham, H., Page, M., Pfaff, B., Goh, E.-J., Modadugu, N., and Boneh, D. On the effectiveness of address-space randomization. In Proceedings of the 11th ACM Conference on Computer and Communications Security (New York, NY, USA, 2004), CCS '04, Association for Computing Machinery, p. 298–307.
- [5] Smith, R. The qualcomm snapdragon x architecture deep dive: Getting to know oryon and adreno x1. AnandTech (2024).
- [6] Narayan, S., Disselkoen, C., Moghimi, D., Cauligi, S., Johnson, E., Gang, Z., Vahldiek-Oberwagner, A., Sahita, R., Shacham, H., Tullsen, D., and Stefan, D. Swivel: Hardening WebAssembly against spectre. In 30th USENIX Security Symposium (USENIX Security 21) (UC San Diego, Aug. 2021), USENIX Association, pp. 1433–1450.
- [7] Newsroom, A. Memory safety with arm memory tagging extension, 2021. Accessed: 2024-04-30
- [8] Oh, C., Lee, S., Qian, C., Koo, H., and Lee, W. Deview: Confining progressive web applications by debloating web apis. In Proceedings of the 38th Annual Computer Security Applications Conference (New York, NY, USA, 2022), ACSAC '22, Association for Computing Machinery, p. 881–895.
- [9] Oleksenko, O., Kuvaiskii, D., Bhatotia, P., Felber, P., and Fetzer, C. Intel mpx explained: A cross-layer analysis of the intel mpx system stack. SIGMETRICS Perform. Eval. Rev. 46, 1 (jun 2018), 111–112.
- [10] Stievenart, Q., De Roover, C., and Ghafari, M. ´ Security risks of porting c programs to webassembly. In Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing (New York, NY, USA, 2022), SAC '22, Association for Computing Machinery, p. 1713–1722.
- [11] Team, C. Hardware-assisted addresssanitizer design documentation, 2024. Accessed: 2024-08-27
- [12] Tice, C., Roeder, T., Collingbourne, P., Checkoway, S., Erlingsson, U., Lozano, L., and Pike, G. Enforcing forward-edge control-flow integrity in gcc & llvm. In Proceedings of the 23rd USENIX Conference on Security Symposium (USA, 2014), SEC'14, USENIX Association, p. 941–955.
- [13] Yan, Y., Tu, T., Zhao, L., Zhou, Y., and Wang, W. Understanding the performance of webassembly applications. In Proceedings of the 21st ACM Internet Measurement Conference (New York, NY, USA, 2021), IMC '21, Association for Computing Machinery, p. 533–549.
- [14] Jangda, A., Powers, B., Berger, E. D., and Guha, A. Not so fast: analyzing the performance of webassembly vs. native code. In Proceedings of the 2019 USENIX Conference on Usenix Annual Technical Conference (USA, 2019), USENIX ATC '19, USENIX Association, p. 107–120.
- [15] Lee, J., Hong, S., and Oh, H. Memfix: static analysis-based repair of memory deallocation errors for c. In Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (New York, NY, USA, 2018), ESEC/FSE 2018, Association for Computing Machinery, p. 95–106
- [16] Chromium. Chromium docs - the rule of 2, 2024. Accessed on 2024-04-20.
- [17] Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., and Bastien, J. Bringing the web up to speed with webassembly. In Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (New York, NY, USA, 2017), PLDI 2017, Association for Computing Machinery, p. 185–200.

Code demo

Thank you!